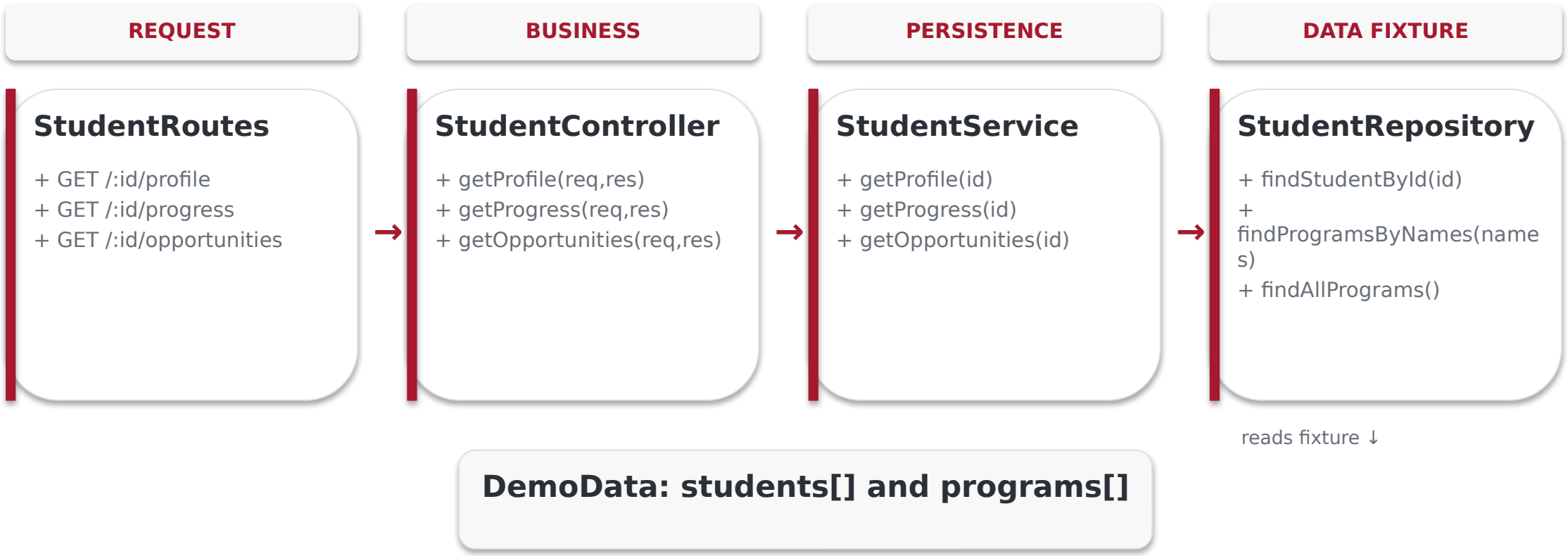
Internal Server Architecture | UML Class Diagram

Chosen architecture: Layered • Current Node.js / Express backend prototype



The repository isolates data access. The current implementation uses fictional in-memory data; a SQL database is not connected.

GitHub / Project Structure

Verified in GitHub main • layered backend merged and documented

```
pathfinder/
├── frontend/UI/            existing browser prototype
└── backend/
    ├── app.js              Express entry point
    ├── routes/students.js
    ├── controllers/studentController.js
    ├── services/studentService.js
    ├── repositories/studentRepository.js
    ├── data/demoData.js
    ├── package.json
    └── package-lock.json
tests/backend.test.js
docs/diagrams/assignment-05-class-diagram.md
```

Request routes/ maps URLs; controllers/ handles incoming requests and JSON responses.

Business services/ calculates course progress, credits, and ranked opportunities.

Persistence repositories/ is the sole access point for student and program records.

Data data/demoData.js supplies temporary fictional fixtures instead of a database.

Feature Mapping | Request Through Layers

Three implemented backend endpoints • All three included backend tests passed

FR-3 Student Information Card

GET /api/students/demo/profile

Route → StudentController → StudentService → StudentRepository → DemoData

getProfile() — Returns sample name, class year, declared majors/minors, school ID.

FR-1 Progress Overview

GET /api/students/demo/progress

Route → StudentController → StudentService → StudentRepository → DemoData

getProgress() — Counts completed/remaining program requirements and earned credits.

FR-2 Opportunity Finder

GET /api/students/demo/opportunities

Route → StudentController → StudentService → StudentRepository → DemoData

getOpportunities() — Ranks up to five eligible major/minor suggestions by course overlap.

Current scope: JSON API prototype, not yet integrated with the frontend, authentication, or a production database.